

INF1430

Projet de fin d'études

UNIVERSITÉ TÉLUQ

Travail noté 2

Date de remise : 14-05-2026

Présenté à : Habib Louafi

Étudiante : Melissa Moya

Numéro d'étudiant : 24332062

Table des matières

1. Introduction	3
2. Description du problème	4
3. Analyse du domaine et démarche de conception.....	5
3.1 Nature du système	5
3.2 Choix de la démarche méthodologique.....	6
3.3 Identification et modélisation des concepts du domaine	7
3.4 Justification des choix de modélisation UML	8
3.5 Synthèse de la démarche.....	9
4. Conception et architecture du système.....	9
4.1 Architecture conceptuelle générale.....	10
4.2 Organisation modulaire et séparation des responsabilités	12
4.3 Organisation des paquets (UML)	13
4.4 Diagramme de classes (UML).....	15
4.5 Diagramme de séquence	18
4.5.1 Scénario nominal d'exécution.....	18
4.5.2 Scénario de test de performance	18
5. Démarche d'ingénierie et choix techniques	19
5.1 Organisation du développement	20
5.2 Environnement technique effectif.....	20
5.3 Gestion de configuration.....	21
5.4 Stratégie de validation fonctionnelle	22
6. Ordre de conception et implémentation	22
7. État d'avancement du projet	24
7.1 Implémentation réalisée.....	25
7.2 Validation fonctionnelle effectuée	27
7.3 Premières expérimentations	28
7.4 Difficultés rencontrées et ajustements	28
8. Conclusion du rapport de mi-parcours	29

Bibliographie	30
Annexe	31
Annexe A – Structure des dossiers et fichiers	31
A.1 domain/	31
A.2 Dossier application/	33
A.3 Dossier scripts/	34
A.4 Dossier data/	34
A.5 Dossier validation/	35
Annexe B – Justification des relations du diagramme de classes	36

1. Introduction

S’inscrivant dans la continuité de la phase de planification, ce document formalise la conception logicielle du moteur expérimental destiné à soutenir la comparaison rigoureuse des algorithmes de chiffrement symétrique. Alors que les fondements théoriques, les hypothèses de recherche et la méthodologie ont été établis lors du premier jalon, l’attention se porte désormais sur la structuration technique de la solution permettant leur mise en œuvre effective.

Cette étape vise à traduire les objectifs méthodologiques en une architecture logicielle modulaire et cohérente, combinant les principes de la conception orientée domaine et de l’approche par objets. L’analyse privilégie l’établissement d’un langage ubiquitaire partagé, permettant une séparation stricte des responsabilités entre les primitives cryptographiques, les modes d’opération et l’orchestration des expérimentations. Une telle structure garantit la maintenabilité et l’extensibilité du système, attributs essentiels pour assurer la répétabilité des mesures de performance ainsi que la validation de l’effet d’avalanche en tant qu’indicateur de sécurité et de robustesse, la diffusion étant assurée par la substitution et la confusion par la permutation, deux mécanismes fondamentaux des algorithmes de chiffrement par blocs.

Ce rapport de mi-parcours présente ainsi les artefacts de conception clés, couvrant les dimensions statiques, dynamiques, logiques et physiques du système. Il a pour objectif de démontrer la convergence entre les abstractions du domaine cryptographique et les décisions architecturales, tout en validant l’état d’avancement de la préparation à l’implémentation pour les phases ultérieures de collecte et d’analyse de données.

L'établissement de ce cadre structurel permet d'aborder avec précision les enjeux de conception et de coordination logicielle présentés dans la section suivante.

2. Description du problème

À l'issue de la définition du cadre expérimental et des paramètres d'analyse, le projet met en évidence un enjeu central de conception logicielle, à savoir la conception d'un système d'analyse comparative capable de soutenir de manière rigoureuse, cohérente et évolutive les expérimentations prévues.

À ce stade, le problème ne porte plus sur le choix des algorithmes ou des modes d'opération, éléments déjà établis lors de la phase de planification. L'attention se déplace désormais vers la structuration d'une architecture logicielle apte à coordonner ces composantes de façon maîtrisée, sans introduire de dépendances inutiles ni de complexité excessive. Une conception inadéquate nuirait non seulement à la lisibilité du système, mais compromettrait également la fiabilité des mesures de performance et de sécurité produites.

L'enjeu réside ainsi dans la capacité à organiser le système de manière à distinguer clairement quatre dimensions critiques :

- **La logique métier**, associant les primitives cryptographiques paramétrables (taille de bloc, taille de clé) et les modes d'opération. Le système doit isoler ces composants afin d'évaluer non seulement leur efficacité computationnelle, mais également leur robustesse cryptographique, notamment à travers l'analyse des mécanismes de diffusion, réalisée par la substitution, et de confusion, réalisée par la permutation. Il convient également de noter que les modes d'opération contribuent eux-mêmes à la sécurité du système, certains modes tels que ECB présentant des vulnérabilités connues tandis que des modes comme GCM offrent en plus une authentification intégrée.
- **L'orchestration des expérimentations**, garantit la neutralité du cadre expérimental. Python est retenu non pour ses performances absolues, mais pour sa lisibilité, la disponibilité de bibliothèques cryptographiques standardisées (PyCryptodome) et la facilité de reproductibilité sur des plateformes hétérogènes. L'architecture vise ainsi à assurer une comparaison équitable à l'aide d'un paramétrage rigoureux, par exemple en validant explicitement les trois opérations successives de chiffrement du 3DES.
- **Les mécanismes d'exécution**, chargés du déclenchement reproductible des processus expérimentaux sur des plateformes hétérogènes, telles que des ordinateurs personnels et des systèmes embarqués de type Raspberry Pi.

- **La gestion des résultats**, responsable du traitement des données expérimentales afin de produire des métriques de performance (débit, temps d'exécution) ainsi que des indicateurs de sécurité, tels que l'effet d'avalanche, considéré ici comme une propriété intrinsèque de la primitive cryptographique et non comme une mesure temporelle.

Cette phase du projet vise ainsi à répondre à la question suivante : *comment concevoir une architecture logicielle modulaire et cohérente permettant de mettre en œuvre des expérimentations de chiffrement paramétrables, tout en garantissant la neutralité des mesures et une séparation stricte entre performance temporelle et robustesse sécuritaire ?* Une fois ces enjeux de coordination et de rigueur technique établis, il devient possible de définir la démarche méthodologique permettant de traduire ces exigences en un modèle conceptuel solide.

3. Analyse du domaine et démarche de conception

La conception du système repose sur une analyse structurée du domaine cryptographique et sur l'adoption d'une démarche de conception adaptée à la nature technique du projet. Cette section présente successivement les caractéristiques du système étudié, la démarche méthodologique retenue et l'identification des principales abstractions du domaine.

3.1 Nature du système

Le système développé est un logiciel technique et algorithmique dont l'objectif principal est de réaliser des expérimentations comparatives rigoureuses sur des schémas de chiffrement symétrique paramétrables. Il est orienté vers le calcul et l'orchestration d'algorithmes, et conçu pour exécuter et évaluer différentes configurations cryptographiques dans un environnement contrôlé, garantissant la répétabilité des mesures.

Contrairement à une application logicielle orientée utilisateur, le système ne comporte ni interface graphique ni scénarios d'utilisation métier. Il agit comme un moteur expérimental, chargé d'assembler, d'exécuter et de comparer des primitives et des modes d'opération. Cette spécialisation permet d'isoler les indicateurs de performance temporelle des indicateurs de sécurité, tels que l'effet d'avalanche, la diffusion et la confusion, en traitant ces derniers comme des propriétés intrinsèques aux algorithmes, indépendantes de l'environnement logiciel.

Bien que la cryptographie symétrique inclue également les chiffrements par flot, ce projet se concentre principalement sur les chiffrements par blocs, tout en intégrant ChaCha20

comme représentant des chiffrements par flot, encapsulé via un StreamMode dédié. Ce choix est motivé par trois facteurs:

1. Conformité au mandat : L'objectif est d'analyser l'impact de paramètres spécifiques aux blocs (taille de bloc, modes de chaînage).
2. Polyvalence des modes : L'utilisation des modes CTR (Counter) et GCM (Galois/Counter Mode, standardisé par le NIST SP 800-38D) permet de transformer un chiffrement par blocs en un mécanisme de type flot, couvrant ainsi fonctionnellement les enjeux de performance liés au traitement continu des données. Le choix de GCM est également motivé par sa conformité aux standards industriels actuels, ce mode offrant conjointement confidentialité et authentification intégrée, contrairement à des modes plus anciens tels que OFB.
3. Analyse de la robustesse : Les algorithmes par blocs offrent un cadre de comparaison plus riche pour l'étude des mécanismes de substitution et de permutation, piliers de la robustesse cryptographique moderne.

Cette nature spécialisée oriente directement les choix de conception, car elle favorise une organisation centrée sur les abstractions du domaine cryptographique et une séparation stricte des responsabilités. En éliminant les couches dédiées à l'interaction utilisateur, l'architecture minimise les biais potentiels et renforce la neutralité du cadre expérimental (assurée par l'abstraction logicielle de Python) nécessaire à la validation rigoureuse des hypothèses de recherche.

Cette définition de la nature technique du système justifie l'adoption d'une approche de conception capable de modéliser avec précision ces exigences algorithmiques, à savoir la démarche orientée domaine.

3.2 Choix de la démarche méthodologique

Au regard de ces caractéristiques techniques, une démarche de conception orientée domaine (Domain-Driven Design), telle que formalisée par (Evans, 2003) et précisée par (Khononov, 2021), a été privilégiée. Ce choix repose sur la stabilité et la rigueur des concepts cryptographiques mobilisés, lesquels sont indépendants de tout scénario d'interaction utilisateur. Dans ce contexte, il est plus productif de débiter la conception par l'établissement d'un langage ubiquitaire partagé et par la structuration des abstractions fondamentales que par la modélisation de cas d'utilisation, souvent peu révélateurs pour un moteur algorithmique.

Les éléments centraux de la conception, soit les primitives, les modes d'opération ainsi que les mécanismes d'orchestration, relèvent de règles formelles établies par des standards

(NIST, FIPS). Une approche fondée sur l'analyse conceptuelle permet de traduire ces règles en invariants logiciels stricts, garantissant que l'architecture technique est le miroir exact du domaine de la cryptographie symétrique.

Cette démarche permet de limiter la sur-ingénierie et de concentrer l'effort sur le cœur logique du système. En isolant les concepts métier des détails d'infrastructure, l'architecture favorise la neutralité du cadre expérimental et produit une solution plus lisible, cohérente et directement alignée avec les objectifs de recherche du projet.

Cette assise méthodologique étant posée, il convient maintenant de matérialiser ces abstractions à travers l'identification et la modélisation des concepts clés du domaine.

3.3 Identification et modélisation des concepts du domaine

L'analyse du domaine a permis d'identifier un ensemble restreint d'abstractions centrales servant de base à la conception du système. Ces abstractions structurent l'architecture autour d'un langage ubiquitaire partagé, garantissant la cohérence entre les modèles de conception et l'implémentation. Les responsabilités sont ainsi clairement réparties :

- La primitive cryptographique : Représente l'algorithme de chiffrement symétrique (ex: AES, DES) indépendamment de son mode d'application. C'est à ce niveau que sont implémentées les propriétés de diffusion (substitution) et de confusion (permutation), et que sont évalués les indicateurs de sécurité intrinsèques comme l'effet d'avalanche, conformément aux principes fondateurs de la cryptographie moderne par blocs (Paar & Pelzl, 2010; Stallings, 2017).
- Le mode d'opération : Décrit la logique de chaînage ou de traitement des blocs de données (ex: CBC, CTR). Il apporte un niveau de sécurité supplémentaire et détermine les possibilités de parallélisation influençant le débit.
- Le moteur de chiffrement : Issu de la composition d'une primitive et d'un mode. Il constitue le service de chiffrement complet, assurant la transformation cohérente des messages en respectant les invariants du domaine (vecteur d'initialisation, nonces, blocs).
- Le contrôleur d'expérimentation : Responsable de l'orchestration. Il configure les moteurs, déclenche les campagnes de tests et collecte de manière neutre les résultats de performance (temps, débit) et de robustesse sécuritaire.

Cette modélisation permet de distinguer strictement les mécanismes cryptographiques de l'orchestration expérimentale, favorisant la neutralité des mesures et l'évolutivité du système.

Une fois ces abstractions clés identifiées et leurs responsabilités définies, il importe de justifier les artefacts UML choisis pour les visualiser et valider leurs interactions.

3.4 Justification des choix de modélisation UML

Les représentations utilisées ont été sélectionnées de manière ciblée selon le principe de sélectivité, afin de décrire les dimensions critiques du système. Conformément aux recommandations de (Fowler, 2003), l'attention se porte sur la communication des structures clés plutôt que sur une complétude exhaustive. Les artefacts retenus sont :

- Un diagramme de classes, utilisé dans une perspective conceptuelle pour établir le langage ubiquitaire du domaine et définir les relations statiques entre les primitives et les modes de chiffrement.
- Un schéma des composantes, offrant une vue d'ensemble de l'organisation modulaire et de la séparation des responsabilités. Ce schéma est crucial pour démontrer visuellement l'isolation de la primitive cryptographique par rapport aux modes d'opération et à l'orchestration expérimentale, garantissant ainsi la neutralité du cadre expérimental exigée pour les mesures.
- Un diagramme de paquets, utilisé pour représenter l'organisation logique du code et valider visuellement les dépendances acycliques entre les couches (orchestration, domaine et analyse), garantissant ainsi le respect du principe de séparation des préoccupations.
- Un diagramme de déploiement, essentiel pour illustrer l'implantation physique du système sur les plateformes d'exécution ciblées. Il documente l'exécution du même artefact logiciel sur des plateformes hétérogènes (PC sous Windows et Raspberry Pi sous Linux), selon un modèle de développement hôtes-cible, justifiant ainsi la validité du protocole multi-plateforme du projet.
- Des diagrammes de séquence ciblés, illustrant d'une part le scénario nominal d'exécution et d'autre part le protocole automatisé de mesure des performances, afin de démontrer la séparation rigoureuse entre la logique cryptographique et les mécanismes de collecte de données
- Un schéma de validation des invariants, matérialisant le mécanisme de contrôle lors de la composition des objets selon le principe de conception par contrat (Design by Contract), afin de prévenir toute configuration cryptographique incohérente avant l'exécution.

À l'inverse, certains artefacts UML ont été délibérément écartés. Les diagrammes de cas d'utilisation ont été jugés non pertinents en l'absence d'acteurs humains interagissant avec le système. Les diagrammes d'états et d'activités, bien qu'utiles pour modéliser des flux complexes, auraient introduit une complexité représentationnelle disproportionnée pour un moteur algorithmique dont le comportement est entièrement déterministe et séquentiel.

L'utilisation conjointe de ces représentations permet d'articuler l'analyse conceptuelle, la structure physique et le comportement dynamique du moteur algorithmique. Cette clarté structurelle constitue le fondement de la synthèse de la démarche présentée ci-après.

3.5 Synthèse de la démarche

La démarche de conception adoptée, ancrée dans la conception orientée domaine, a permis de traduire la complexité de la cryptographie symétrique en un ensemble restreint et rigoureux d'abstractions logiques. Cette formalisation progressive garantit que l'architecture technique restera le miroir fidèle des principes de diffusion et de confusion propres au domaine.

En privilégiant l'établissement d'un langage ubiquitaire et la séparation stricte entre les primitives et leur orchestration, cette démarche assure la construction d'un système modulaire et neutre. Elle prépare ainsi une architecture capable d'évaluer de manière isolée l'impact des paramètres sur les performances et la sécurité, conformément aux objectifs expérimentaux du projet. Cette cohérence entre l'analyse et la conception constitue l'assise nécessaire pour structurer l'architecture technique détaillée dans la section suivante.

4. Conception et architecture du système

À la suite de l'analyse du domaine et de la définition de la démarche de conception, cette section formalise la structure logicielle du moteur expérimental. Elle décrit l'organisation des composants ainsi que le comportement dynamique associé, afin de démontrer comment les abstractions identifiées collaborent pour assurer une comparaison rigoureuse des algorithmes.

La conception privilégie une architecture robuste et modulaire, fondée sur une séparation explicite des responsabilités. Cette section s'appuie sur des représentations UML complémentaires couvrant les dimensions critiques du système :

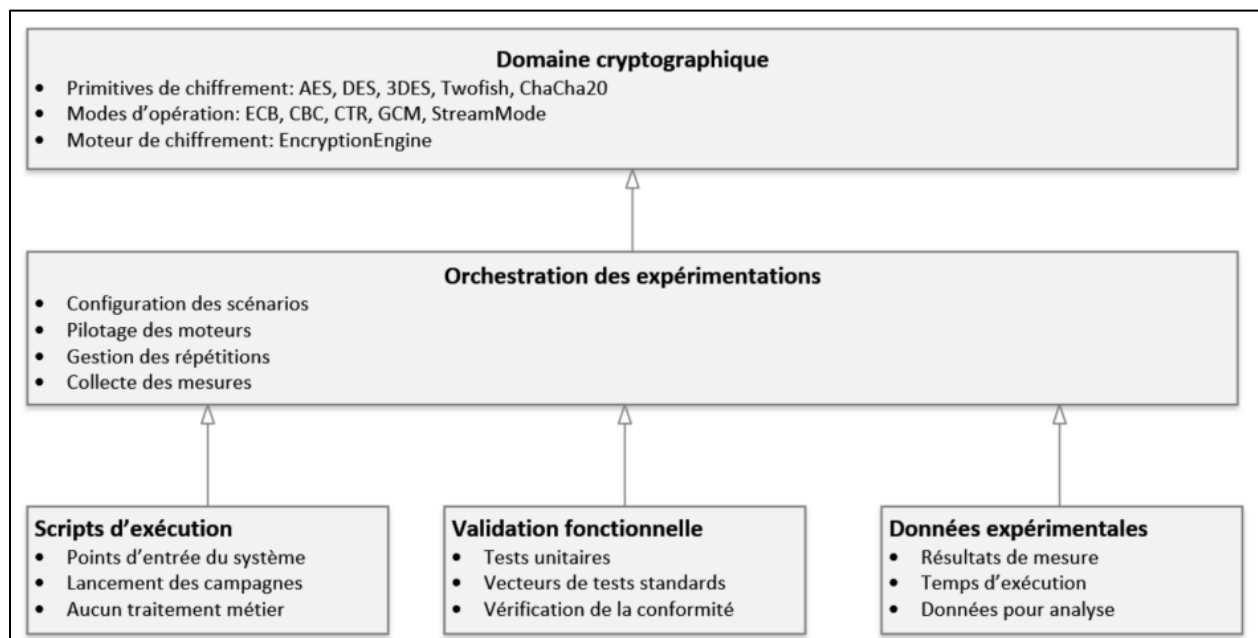
- Une vue architecturale générale (composantes) ;
- Un diagramme de déploiement (physique) ;
- Un diagramme de paquets (logique) ;
- Un diagramme de classes (statique) ;

- Des diagrammes de séquence (dynamique).

4.1 Architecture conceptuelle générale

L'architecture conceptuelle offre une vue d'ensemble du système en identifiant les principaux blocs logiques et leurs interactions. Elle vise à clarifier la répartition des responsabilités fonctionnelles, garantissant que la logique cryptographique demeure strictement isolée des mécanismes de mesure et d'orchestration expérimentale.

Figure 1 — Schéma des composantes du système



Ce schéma illustre l'organisation modulaire de la solution autour de quatre blocs critiques :

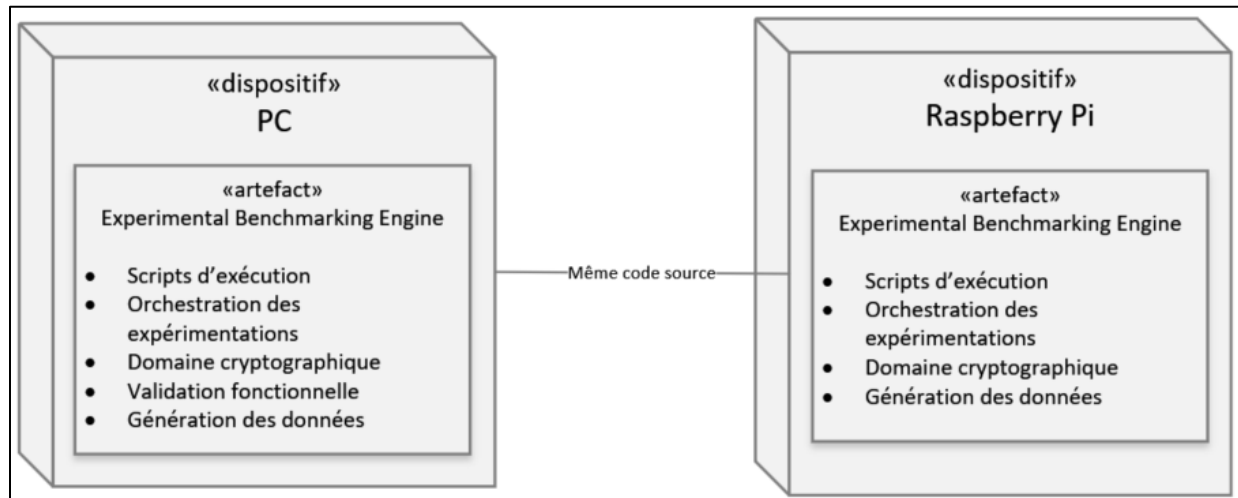
- **Bloc de chiffrement** : Ce bloc encapsule les primitives cryptographiques fondamentales du projet, soit l'AES, le DES, le 3DES (caractérisé par sa triple opération successive), Twofish ainsi que ChaCha20, chiffre de flux encapsulé via le StreamMode dédié. Il fournit les fonctionnalités de chiffrement et de déchiffrement de manière autonome, en restant strictement indépendant des modes d'opération afin de dissocier les algorithmes des mécanismes de traitement des données. Cette isolation est cruciale pour évaluer les propriétés intrinsèques de chaque primitive, notamment ses mécanismes de diffusion (substitution) et de confusion (permutation), piliers de la robustesse cryptographique.
- **Bloc de contrôle des modes** : ce bloc agit comme une couche d'abstraction chargée d'appliquer les différents modes d'opération (ECB, CBC, CTR, GCM et StreamMode)

aux primitives cryptographiques. Il gère rigoureusement l'enchaînement des blocs ainsi que l'utilisation des vecteurs d'initialisation ou des nonces. Cette séparation permet d'analyser comment chaque mode apporte un niveau de sécurité supplémentaire ou comment un mode faible comme l'ECB peut rendre un algorithme vulnérable.

- **Bloc d'expérimentation** : ce bloc assure l'orchestration des campagnes d'analyse comparative. Il est responsable de la configuration des tests en fonction des paramètres variables tels que les tailles de clés, les tailles de blocs et les messages. Sa fonction principale est de garantir la répétabilité des expérimentations et la neutralité des mesures en coordonnant les interactions entre les couches logiques et les plateformes matérielles.
- **Bloc d'analyse** : le bloc d'analyse traite les données brutes issues des tests pour produire des indicateurs exploitables, en distinguant strictement la performance temporelle de la robustesse sécuritaire. Il permet le calcul de métriques d'efficacité, comme le débit, ainsi que l'analyse de la force cryptographique via l'effet d'avalanche. Cette mesure, calculée par la distance de Hamming, est traitée comme un indicateur de sécurité intrinsèque à l'algorithme et non comme une mesure de temps.

Afin de compléter cette vue fonctionnelle, un diagramme de déploiement illustre l'implantation physique du système sur les différentes plateformes matérielles.

Figure 2 — Diagramme de déploiement du système expérimental



Ce diagramme documente le modèle de développement hôte-cible. Le moteur expérimental est conçu sur une station de travail (PC sous Windows), puis déployé sans modification sur un environnement aux ressources limitées (Raspberry Pi sous Linux). Cette approche met en évidence l'exécution d'un même artefact logiciel dans des environnements hétérogènes, validant ainsi la neutralité du cadre expérimental et la portabilité de la solution Python retenue, laquelle fait abstraction des optimisations de bas niveau propres au matériel.

Le diagramme de déploiement est utilisé ici dans une perspective conceptuelle et vise à illustrer la séparation logique entre le moteur expérimental et les plateformes d'exécution considérées. Il ne décrit ni une configuration matérielle détaillée ni des optimisations spécifiques, celles-ci étant volontairement abstraites afin de préserver la neutralité du cadre expérimental.

Cette structuration en blocs logiques et les responsabilités ainsi définies permettent désormais de formaliser les relations statiques entre les objets du système au sein du diagramme de classes.

4.2 Organisation modulaire et séparation des responsabilités

L'architecture du système repose sur une organisation modulaire rigoureuse, ancrée dans les principes fondamentaux de la programmation orientée objet (POO). Cette approche permet d'isoler les responsabilités associées à chaque composant selon le principe de séparation des préoccupations, garantissant un faible couplage entre la logique métier cryptographique, les mécanismes d'orchestration et les processus d'exécution, tout en assurant une forte cohésion interne de chaque module.

Chaque module remplit un rôle bien défini et interagit avec les autres composants par des interfaces explicites. Cette approche offre trois avantages critiques pour la réussite du projet. Premièrement, l'évolutivité et la flexibilité du système permettent d'ajouter ou de modifier un algorithme de chiffrement, un mode d'opération ou un scénario expérimental sans remettre en cause l'intégrité de l'ensemble du système. Cette propriété est essentielle pour soutenir la nature incrémentale de la phase d'implémentation. Deuxièmement, la neutralité des mesures est assurée par l'isolation stricte de la primitive cryptographique des couches d'orchestration, minimisant ainsi les biais logiciels. Cela garantit que les variations de performance ou de robustesse observées résultent uniquement des algorithmes testés et non de la structure du moteur expérimental. Troisièmement, la précision du paramétrage est facilitée par la structure modulaire, qui permet le paramétrage granulaire exigé (tailles de blocs, longueurs de clés, vecteurs d'initialisation), permettant ainsi de valider rigoureusement les hypothèses expérimentales sur des plateformes hétérogènes.

Cette architecture modulaire a été retenue en opposition à une approche monolithique, dans laquelle les primitives cryptographiques, les modes d'opération et les mécanismes de mesure seraient étroitement couplés. Une telle approche aurait accru la complexité du système, réduit sa lisibilité et introduit des biais méthodologiques lors des mesures de performance. Le découplage retenu favorise au contraire une analyse isolée des algorithmes et des modes, tout en améliorant la maintenabilité et l'évolutivité du moteur expérimental.

Sur le plan des patrons de conception, l'architecture s'inspire du patron Stratégie (Strategy Pattern), dans lequel le comportement variable, soit le mode d'opération, est encapsulé derrière une interface commune et peut être substitué dynamiquement sans modifier le contexte qui l'utilise, soit le moteur de chiffrement. Ce patron garantit que l'ajout d'un nouveau mode d'opération se limite à l'implémentation d'une nouvelle classe conforme à l'interface `OperationMode`, sans aucun impact sur les couches supérieures du système.

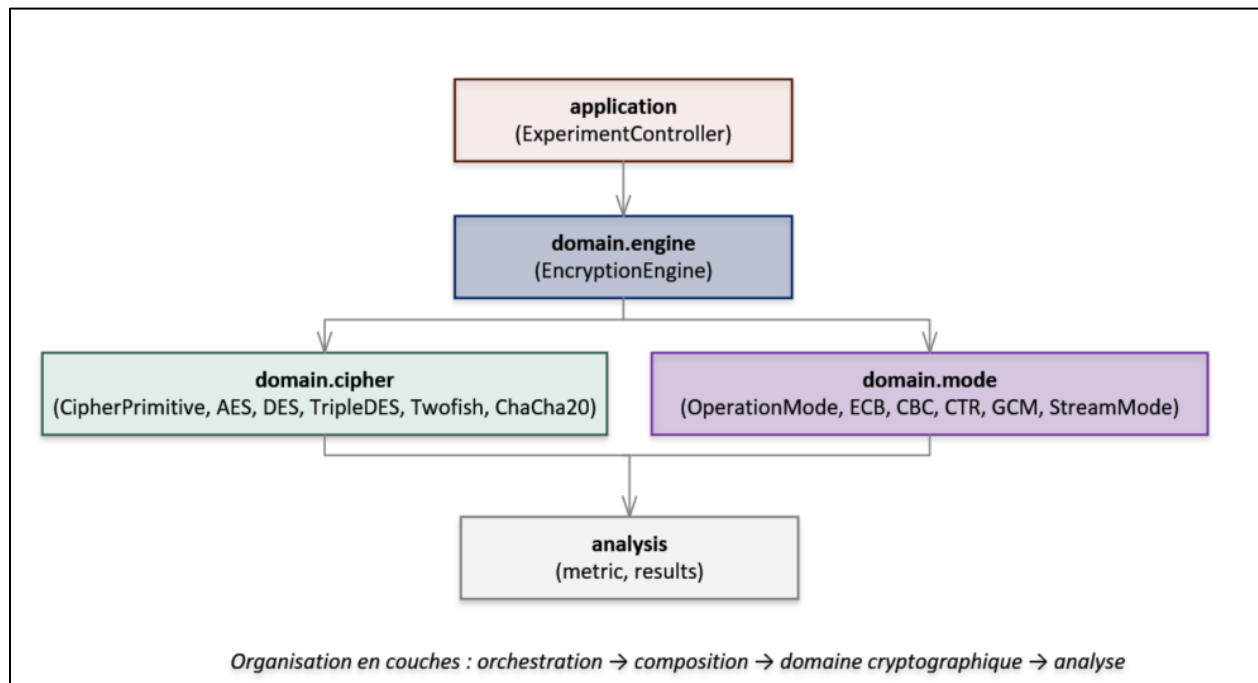
Cette extensibilité est concrètement illustrée par l'intégration de la classe `StreamMode` et de la primitive `ChaCha20`, toutes deux implémentées dans le cadre de ce projet. En se conformant à l'interface `OperationMode`, `StreamMode` permet d'intégrer `ChaCha20` sans modification de `EncryptionEngine` ni du contrôleur d'expérimentation, démontrant ainsi concrètement la portée du patron Stratégie retenu. Cette conception répond directement à l'observation formulée lors du premier jalon quant à l'opportunité d'inclure les chiffrements par flot dans le périmètre comparatif.

Un tel découplage constitue le cadre logique nécessaire à la formalisation de l'architecture statique du système, présentée dans les sections suivantes à travers les diagrammes de paquets et de classes.

4.3 Organisation des paquets (UML)

Un diagramme de paquets est utilisé afin de représenter l'organisation logique du code et les dépendances principales entre les modules du système. Cette vue intermédiaire complète l'architecture conceptuelle en montrant comment les responsabilités identifiées sont regroupées en ensembles cohérents, sans entrer dans le détail des classes ou de l'implémentation.

Figure 3 — Diagramme de paquets du système



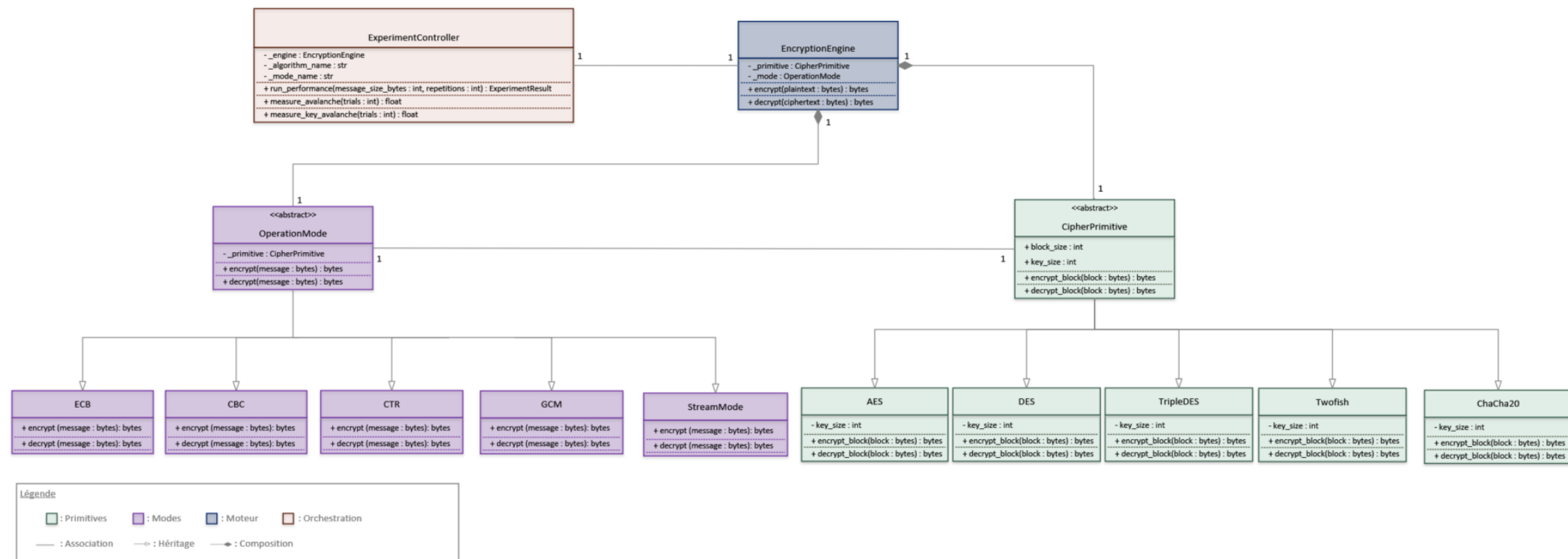
Ce diagramme met en évidence une organisation en couches du système, distinguant l'orchestration, la composition, le domaine cryptographique et l'analyse. Les dépendances sont strictement orientées vers le bas, garantissant que le cœur cryptographique demeure indépendant des mécanismes d'exécution et d'analyse. Cette structuration renforce le faible couplage, la maintenabilité du système et la neutralité du cadre expérimental. Le paquet `domain.cipher` regroupe les cinq primitives implémentées, soit AES, DES, TripleDES, Twofish ainsi que ChaCha20, ce dernier étant un chiffre de flux encapsulé via la classe `StreamMode` du paquet `domain.mode`. L'architecture modulaire retenue permet l'ajout futur de nouvelles primitives sans nécessiter de modification structurelle du système.

4.4 Diagramme de classes (UML)

Le diagramme de classes formalise la structure statique du système en représentant les principales abstractions du domaine cryptographique et les relations qui les lient. Dessiné d'une perspective conceptuelle, il permet d'établir un vocabulaire rigoureux fondé sur le langage ubiquitaire du projet. Il met en évidence l'organisation des primitives cryptographiques, des modes d'opération et leur composition au sein du moteur de chiffrement.

Cette représentation illustre la séparation stricte des responsabilités entre la logique algorithmique pure, les mécanismes de composition et l'orchestration des expérimentations, constituant ainsi le socle de l'architecture avant l'analyse de son comportement dynamique.

Figure 4 — Diagramme de classes du système



La modélisation retenue reflète directement les choix conceptuels du projet :

- **Abstractions distinctes** : Les primitives cryptographiques et les modes d'opération sont modélisés comme des entités indépendantes. Cette isolation garantit la neutralité des mesures en permettant d'évaluer l'impact sécuritaire de chaque composant de manière isolée.
- **Héritage et substituabilité** : L'utilisation de l'héritage permet de représenter les variantes concrètes des algorithmes tout en garantissant leur substituabilité au sein du système. Ainsi, le moteur peut traiter n'importe quelle primitive conforme à l'interface définie sans modification de son code source.
- **Composition et faible couplage** : Le moteur de chiffrement assure la composition dynamique d'une primitive et d'un mode. La référence directe à la primitive au sein de l'EncryptionEngine est une dépendance de validation : elle sert exclusivement à vérifier l'invariant de cohérence lors de la construction, et non à exécuter des opérations cryptographiques, qui restent entièrement déléguées au mode. Ce choix architectural exprime les dépendances structurelles nécessaires sans introduire de couplage excessif, facilitant ainsi l'ajout futur de nouveaux algorithmes.
- **Paramétrisation des tailles de clés** : La modélisation distingue explicitement les tailles de clés supportées par chaque primitive. AES supporte des clés de 128, 192 ou 256 bits ; 3DES opère avec des clés de 16 octets (112 bits effectifs, variante 2 clés) ou 24 octets (168 bits effectifs, variante 3 clés), les deux variantes sont évaluées dans ce projet afin de mesurer l'impact de la longueur de clé sur les performances. ChaCha20, chiffre de flux, utilise exclusivement une clé de 256 bits et est encapsulé via un StreamMode dédié. Cette granularité permet d'évaluer l'impact de la longueur de clé sur les performances et la robustesse sécuritaire, constituant l'un des axes expérimentaux centraux du projet.

Tableau 1 — Synthèse des tailles de clés par algorithme

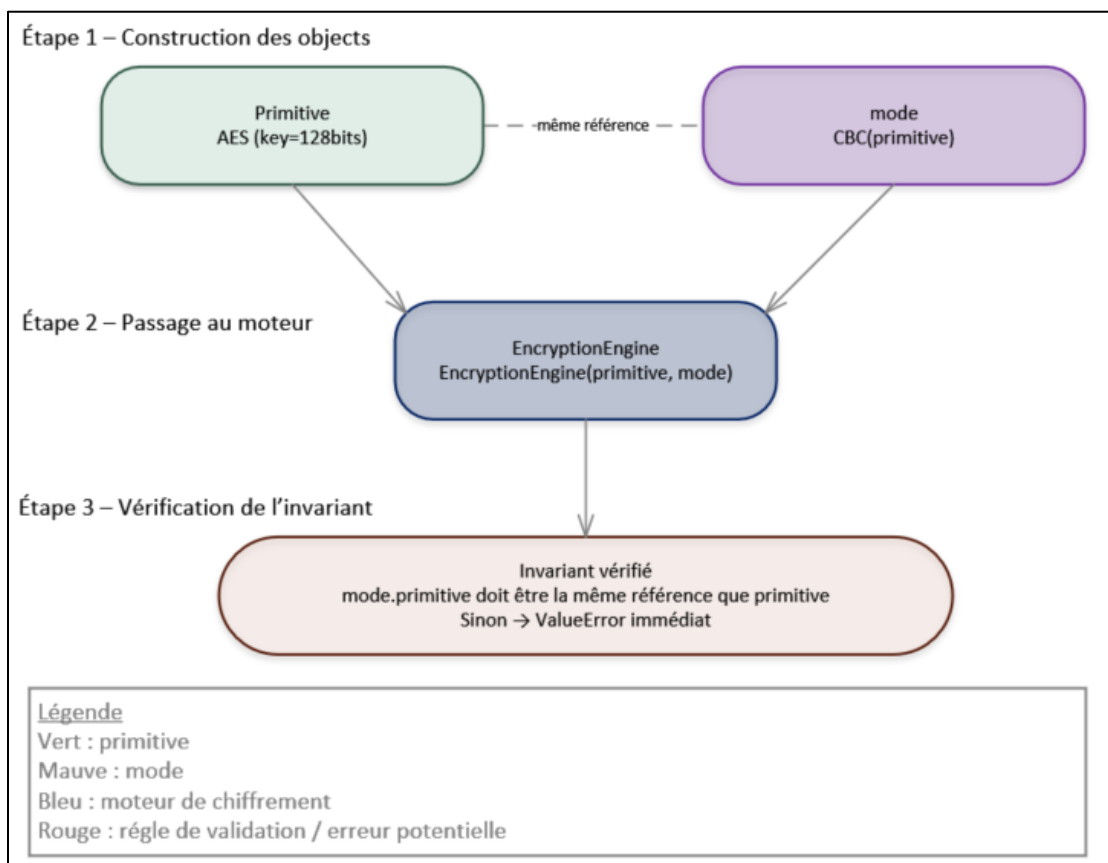
Algorithme	Architecture	Taille de bloc	Tailles de clé supportées
DES	Feistel	64 bits	56 bits (effectifs)
3DES	Feistel	64 bits	112 bits (2 clés) / 168 bits (3 clés)
AES	Substitution-permutation	128 bits	128, 192, 256 bits
Twofish	Feistel	128 bits	128, 192, 256 bits
ChaCha20	Flux (ARX)	—	256 bits

Au-delà de la structure statique, le diagramme met également en évidence un mécanisme de validation des invariants du domaine. Le moteur de chiffrement vérifie explicitement, lors

de sa construction, que le mode d'opération reçu est bien associé à la même primitive cryptographique que celle configurée pour le moteur. Cette vérification garantit la cohérence de la composition et prévient toute configuration incohérente qui pourrait compromettre l'intégrité des résultats expérimentaux. Ce type de contrat logiciel, inspiré de la conception par contrat, constitue une pratique d'ingénierie rigoureuse qui renforce la fiabilité du système.

La figure 5 illustre ce mécanisme de validation. Lors de la construction du moteur, l'objet primitif est d'abord instancié, puis transmis au mode d'opération. L'EncryptionEngine vérifie ensuite que le mode reçu référence bien ce même objet primitif, garantissant ainsi la cohérence de la composition.

Figure 5 — Validation de l'invariant de composition dans EncryptionEngine



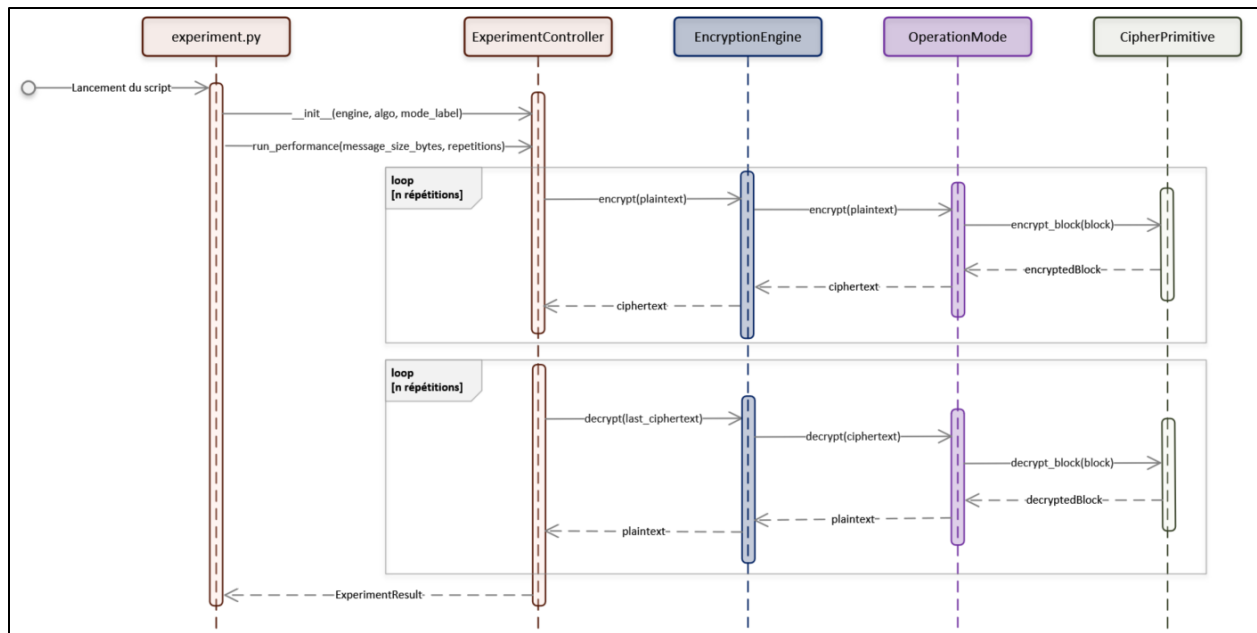
Si le diagramme de classes permet de formaliser la structure statique du système et la répartition des responsabilités entre les abstractions du domaine, il ne rend pas compte du comportement dynamique du moteur expérimental lors de son exécution. Il est donc nécessaire de compléter cette vue par des diagrammes de séquence afin d'illustrer l'orchestration des composants et la mise en œuvre effective des protocoles expérimentaux.

4.5 Diagramme de séquence

4.5.1 Scénario nominal d'exécution

Afin de compléter la vue statique, un diagramme de séquence est utilisé pour représenter le comportement dynamique du système lors de l'exécution d'une expérimentation de chiffrement. Dessiné selon le principe de sélectivité, il illustre un scénario nominal, depuis le déclenchement du banc d'essai jusqu'à l'exécution des algorithmes cryptographiques.

Figure 6 — Diagramme de séquence du scénario nominal d'exécution

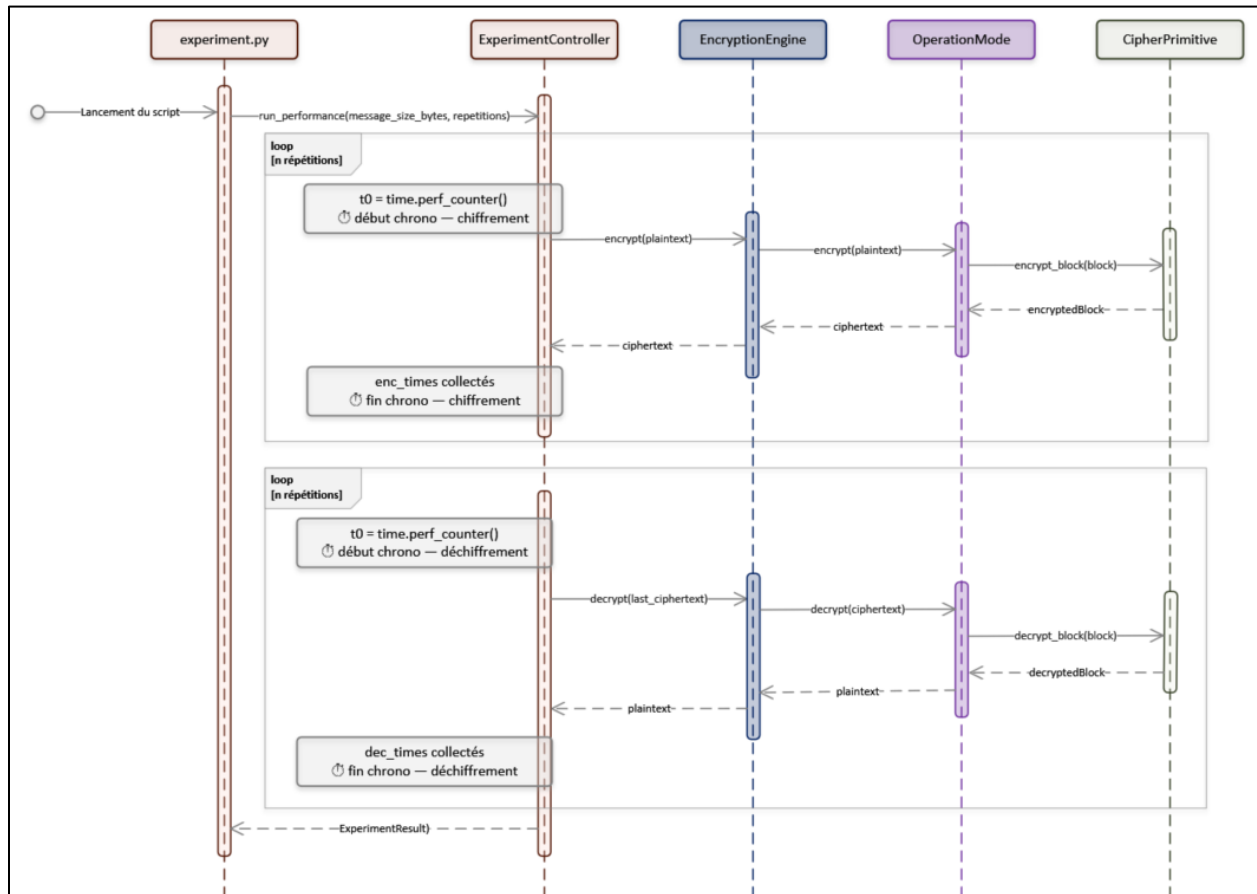


Ce diagramme met en évidence l'enchaînement des interactions entre le script de lancement (`experiment.py`), le contrôleur d'expérimentation (`ExperimentController`) et les composants du domaine (`EncryptionEngine`, `OperationMode`, `CipherPrimitive`). Le script instancie d'abord le contrôleur via `__init__`, puis déclenche `run_performance()`. Deux boucles de `n répétitions` sont exécutées séquentiellement : une pour le chiffrement, une pour le déchiffrement. L'`ExperimentResult` n'est retourné qu'après la complétion des deux boucles.

4.5.2 Scénario de test de performance

Afin de compléter l'analyse du comportement dynamique, un diagramme de séquence spécifique illustre le protocole de mesure des performances. Cette représentation met en évidence le mécanisme de chronométrage intégré à `ExperimentController`, qui encadre chaque boucle d'exécution afin de mesurer exclusivement le temps du cœur algorithmique.

Figure 7 — Diagramme de séquence du test de performance



Ce diagramme montre que le chronométrage est géré directement par ExperimentController via `time.perf_counter()`, sans classe dédiée séparée. Le chrono est démarré et arrêté à chaque itération, et les temps individuels sont collectés dans une liste. Cette organisation garantit la neutralité des mesures et la séparation rigoureuse entre orchestration expérimentale, calcul cryptographique et collecte des performances. Une fois les n répétitions complétées, ExperimentController retourne un objet `ExperimentResult` contenant les métriques agrégées, dont l'analyse sera présentée dans le rapport final.

5. Démarche d'ingénierie et choix techniques

Cette section présente les décisions d'ingénierie et les choix techniques retenus pour la mise en œuvre de l'architecture logicielle. Elle décrit la traduction concrète de la conception en une organisation effective du développement, garantissant que chaque composant implémenté reste fidèle aux abstractions du domaine.

L'accent est porté sur l'organisation modulaire du système, l'environnement technique utilisé (sélectionné spécifiquement pour assurer la neutralité du cadre expérimental) et les

mécanismes de validation fondés sur des standards reconnus. Ces choix techniques constituent le socle nécessaire pour assurer la fiabilité, la portabilité et la reproductibilité des mesures de performance et de sécurité, lesquelles sont des conditions indispensables à la validité des expérimentations multi-plateformes impliquant le PC et le Raspberry Pi.

5.1 Organisation du développement

Le développement du système repose sur une organisation modulaire qui matérialise le principe de séparation des préoccupations (*Separation of Concerns*). Cette structuration, qui reflète directement les choix architecturaux présentés précédemment, vise à assurer un faible couplage entre les composants tout en maintenant une forte cohésion interne, facilitant ainsi l'évolution incrémentale et la maintenance du code.

Les algorithmes de chiffrement (DES, 3DES, AES, Twofish et ChaCha20) sont conçus sous forme de modules autonomes. Ils sont strictement indépendants des modules responsables des modes d'opération (ECB, CBC, CTR, GCM et StreamMode), lesquels agissent comme des couches d'abstraction interchangeables. Cette séparation est essentielle pour garantir la neutralité des mesures :

1. Isolation des indicateurs : elle permet d'évaluer les propriétés de diffusion (substitution) et de confusion (permutation) intrinsèques à chaque primitive (comme les trois opérations du 3DES) sans qu'elles soient altérées par la logique du mode d'opération.
2. Flexibilité expérimentale : elle autorise l'ajout, la comparaison ou le remplacement d'un algorithme ou d'un mode sans impact sur l'orchestration globale du système, une propriété indispensable dans un cadre de recherche.
3. Reproductibilité multi-plateforme : cette modularité permet d'exécuter le même cœur logique de manière identique sur PC et sur Raspberry Pi, assurant que les écarts de performance mesurés résultent uniquement des contraintes matérielles et non de variations dans l'implémentation du logiciel.

Cette organisation structurée du code source constitue le socle logique nécessaire à la mise en œuvre de l'environnement technique détaillé dans la sous-section suivante.

5.2 Environnement technique effectif

La solution est développée en Python, un langage retenu à l'issue d'une évaluation comparative des alternatives disponibles. C++, bien que privilégié en cryptographie pour ses performances brutes, introduit une complexité de gestion mémoire et une dépendance aux optimisations du compilateur qui auraient nui à la reproductibilité des mesures entre

plateformes. Java, quant à lui, impose une verbosité architecturale et une machine virtuelle (JVM) dont le comportement d'optimisation dynamique (JIT) constitue une source de biais difficilement contrôlable dans un cadre expérimental rigoureux. Python a donc été retenu pour sa capacité à abstraire uniformément les optimisations matérielles de bas niveau, garantissant des conditions d'exécution comparables entre les plateformes testées, tout en offrant une lisibilité et une modularité adaptées à la mise en œuvre de l'architecture orientée domaine. Cette neutralité tient au fait que l'interpréteur Python applique le même surcoût d'exécution (overhead uniforme) sur toutes les plateformes testées, contrairement à C/C++ dont le compilateur génère des optimisations spécifiques à l'architecture cible, ou à la JVM Java dont le compilateur JIT introduit une variabilité difficilement contrôlable.

Les implémentations cryptographiques s'appuient principalement sur la bibliothèque PyCryptodome pour les algorithmes supportés, garantissant l'utilisation de primitives conformes aux standards reconnus. Les paramètres expérimentaux, tels que les tailles de clés, les tailles de blocs et les modes d'opération, restent pleinement maîtrisés.

Il convient toutefois de préciser que PyCryptodome délègue les opérations cryptographiques critiques à des routines C optimisées. Cette caractéristique ne compromet pas la neutralité, dans la mesure où ces routines binaires sont identiques sur les deux plateformes testées : la comparaison porte ainsi sur le comportement du même code natif soumis à des contraintes matérielles différentes, ce qui constitue précisément l'objet de la mesure.

Pour les algorithmes ne disposant pas d'un support direct au sein de cette bibliothèque, notamment Twofish, des implémentations compatibles ont été intégrées de manière cohérente avec l'architecture du système, afin de préserver l'homogénéité des composants et la comparabilité des expérimentations.

Au-delà du choix des bibliothèques, la maîtrise du cycle de vie logiciel impose un contrôle rigoureux des évolutions par le biais d'un dispositif de gestion de configuration.

5.3 Gestion de configuration

La gestion des évolutions du code source repose sur Git, un système de contrôle de version distribué hébergé sur GitHub. Ce dispositif assure la traçabilité complète des modifications apportées au système, permettant de documenter chaque décision d'implémentation et de revenir à un état antérieur stable en cas de régression.

L'organisation du dépôt reflète directement la structure modulaire de l'architecture, où chaque couche du système, soit le domaine cryptographique, l'orchestration, les scripts d'exécution, la validation et les données, correspond à un répertoire distinct, facilitant la

navigation et la compréhension du code. Cette structure garantit également que les modifications apportées à une primitive cryptographique ou à un mode d'opération restent isolées et n'affectent pas les autres composants du système.

Cette traçabilité rigoureuse constitue une condition indispensable à la reproductibilité des expérimentations, en assurant que les résultats produits sur PC et sur Raspberry Pi correspondent toujours au même état du code source.

5.4 Stratégie de validation fonctionnelle

Préalablement à toute mesure de performance, le système suit une phase de validation fonctionnelle rigoureuse afin de garantir la conformité absolue des résultats cryptographiques, principe central de toute évaluation rigoureuse en cryptographie (Katz & Lindell, 2007). Cette étape de vérification vise à s'assurer que le logiciel est construit fidèlement aux spécifications théoriques avant d'en évaluer l'efficacité.

Cette validation repose sur l'utilisation de vecteurs de tests standards, spécifiquement les tests de réponse connue (Known Answer Tests). Les primitives retenues (AES, DES, 3DES, Twofish et ChaCha20) sont confrontées aux valeurs de référence des standards reconnus lorsque disponibles, soit le FIPS 197 pour l'AES, le SP 800-17 du NIST pour le DES, le SP 800-67 pour le 3DES, et la RFC 8439 pour ChaCha20. La conformité des modes d'opération par blocs (ECB, CBC et CTR) est validée selon les spécifications de la publication SP 800-38A du NIST, tandis que le mode GCM est validé selon le SP 800-38D. Le mode StreamMode, associé à ChaCha20, est couvert par la validation RFC 8439 de cette primitive. Pour le 3DES, la validation confirme le respect strict de la séquence des trois opérations successives propre à cet algorithme.

Les évaluations de performance temporelle et les analyses de robustesse sécuritaire (effet d'avalanche) ne sont menées qu'après cette confirmation technique. Ce protocole garantit que les mesures reposent sur des implémentations certifiées mathématiquement, assurant ainsi la crédibilité et l'intégrité des résultats comparatifs multi-plateformes obtenus.

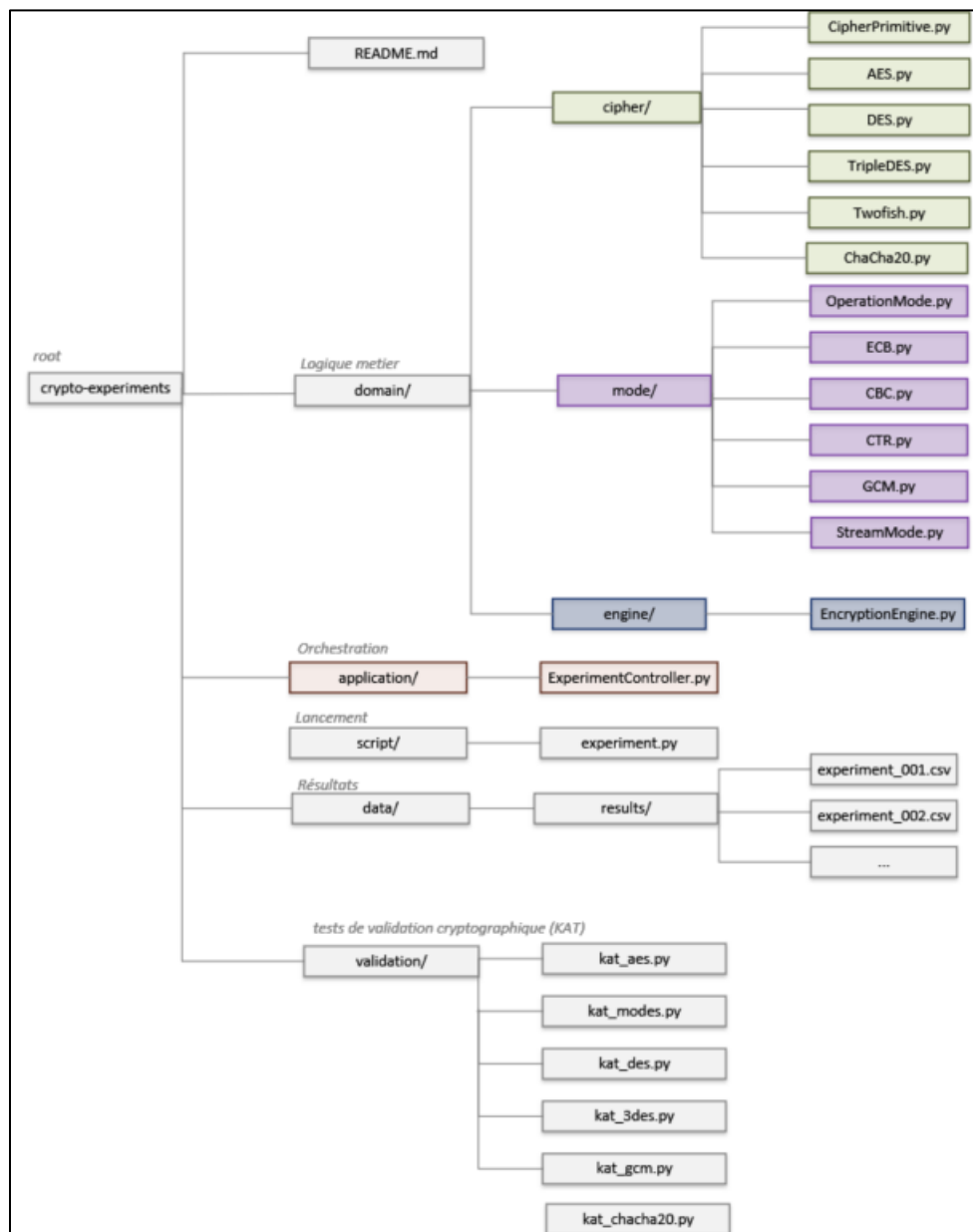
6. Ordre de conception et implémentation

Afin d'assurer une progression cohérente entre l'analyse conceptuelle, la conception logicielle et l'implémentation du système, le développement a été conduit selon un ordre de conception structuré. Cet ordre vise à établir successivement les fondations logiques du système avant l'introduction des mécanismes d'orchestration, de validation et de production des données expérimentales.

La figure 8 présente l'organisation générale du dépôt et met en évidence la structuration du système en couches distinctes. Elle illustre la séparation stricte entre la logique métier du

domaine cryptographique, l'orchestration des expérimentations, les scripts de lancement, les mécanismes de validation et la génération des données finales.

Figure 8 — Organisation générale du projet et séparation des responsabilités



Sur la base de cette organisation, le tableau 2 synthétise l'ordre selon lequel les principaux composants ont été conçus et intégrés. Cette progression repose sur le principe de design tactique, où chaque couche s'appuie sur les abstractions établies précédemment afin de limiter les dépendances et de garantir la stabilité du système. La lecture verticale de cette figure reflète également l'ordre logique d'implémentation retenu, allant des abstractions du

domaine cryptographique vers les couches d'orchestration, de validation et de génération des données expérimentales.

Tableau 2 — Ordre de conception et d'implémentation des composants

#	Composant / Fichier(s)	Justification
1	Couche domaine	Cette couche définit la logique cryptographique centrale (le cœur du logiciel). Elle encapsule les primitives et leurs propriétés de diffusion et de confusion. Elle constitue le socle dont dépendent tous les autres composants.
2	Couche application	Une fois les primitives définies, cette couche orchestre les expérimentations. Elle gère les répétitions et la collecte des mesures sans interagir avec les détails internes des algorithmes, garantissant la neutralité du moteur.
3	Scripts d'exécution	Points d'entrée du programme, ils déclenchent les campagnes expérimentales. Leur séparation de la logique métier permet d'assurer la portabilité du système sur PC et Raspberry Pi.
4	Validation et tests	Phase critique utilisant les vecteurs de référence (KAT) issus des standards NIST et de la RFC 8439. Elle vérifie la correction fonctionnelle avant toute mesure de performance afin de garantir que les résultats produits sont crédibles et exempts d'erreurs d'implémentation.
5	Génération des données	Les données sont produites en dernier, une fois l'implémentation stabilisée. Elles distinguent les mesures de performance temporelle des indicateurs de robustesse sécuritaire (effet d'avalanche).

Les descriptions détaillées des fonctions associées à chacun de ces composants sont fournies en annexe A. Cet ordre de conception permet de relier explicitement les choix architecturaux à l'implémentation effective, assurant ainsi la fiabilité des résultats produits.

Cette planification rigoureuse de l'implémentation permet maintenant de dresser un état d'avancement précis du projet et d'identifier les jalons déjà franchis.

7. État d'avancement du projet

Cette section présente l'état d'avancement du projet à la date de rédaction du rapport. Elle précise les éléments de conception finalisés, ceux actuellement en cours de préparation, ainsi que les travaux planifiés pour les phases suivantes. Ce bilan permet de confirmer la

faisabilité du banc d'essai expérimental dans le respect de l'échéancier initial et des exigences de rigueur scientifique.

Arrivé au jalon du rapport de mi-parcours, le projet a complété la formalisation de son architecture logicielle et de ses abstractions du domaine. La structuration du [dépôt Git](#) et l'organisation modulaire du code sont désormais établies, offrant un cadre stable pour le développement. L'effort de cette phase a porté sur la formalisation exhaustive de l'architecture logicielle et la structuration modulaire du dépôt, constituant le cadre technique dans lequel s'inscriront les mesures de performance et de robustesse sécuritaire (effet d'avalanche) lors des phases expérimentales.

7.1 Implémentation réalisée

À ce stade du projet, l'architecture logicielle et les abstractions du domaine ont été formalisées de manière exhaustive, marquant l'aboutissement du design tactique. L'ossature technique du système est intégralement déployée au sein du dépôt Git, offrant un cadre de développement rigoureux.

Cette organisation repose sur une structure de fichiers modulaire qui matérialise les choix architecturaux présentés précédemment :

1. Cœur logique (domaine) : les interfaces et les définitions des classes pour les primitives cryptographiques et les modes d'opération sont établies, garantissant une intégration cohérente de l'ensemble des composants du domaine.
2. Orchestration (application) : le contrôleur d'expérimentation est configuré pour piloter les moteurs de chiffrement de manière agnostique, respectant la séparation entre la logique métier et la gestion des tests.
3. Scripts et données : les points d'entrée du système ainsi que l'infrastructure de stockage des résultats sont opérationnels, préparant ainsi la phase de collecte des données expérimentales.

La mise en place de cette architecture assure que les phases expérimentales resteront fidèles aux abstractions identifiées. Ce travail sécurise la portabilité du code et garantit la neutralité du banc d'essai lors des exécutions sur PC et Raspberry Pi.

Afin d'illustrer concrètement la traduction des abstractions du domaine en code, les extraits suivants présentent les composants centraux de l'architecture telle qu'elle est déployée à ce stade.

Extrait 1 — Classe abstraite CipherPrimitive

Cette classe définit le contrat d'interface commun à toutes les primitives cryptographiques. Elle garantit que chaque algorithme expose les mêmes propriétés (`block_size`, `key_size`) et les mêmes opérations (`encrypt_block`, `decrypt_block`), permettant au moteur de chiffrement de les traiter de manière uniforme et interchangeable.

```
from abc import ABC, abstractmethod

class CipherPrimitive(ABC):
    """Interface commune pour les primitives de chiffrement symétrique par blocs."""

    @property
    @abstractmethod
    def block_size(self) -> int:
        """Taille du bloc en octets (ex. 16 pour AES, 8 pour DES/3DES)."""

    @property
    @abstractmethod
    def key_size(self) -> int:
        """Taille de la clé en octets utilisée par cette instance."""

    @abstractmethod
    def encrypt_block(self, block: bytes) -> bytes:
        """Chiffre exactement un bloc de texte en clair."""

    @abstractmethod
    def decrypt_block(self, block: bytes) -> bytes:
        """Déchiffre exactement un bloc de texte chiffré."""
```

Code source complet : [domain/cipher/CipherPrimitive.py](#)

Extrait 2 — Classe abstraite OperationMode

Cette classe définit le contrat d'interface pour tous les modes d'opération. Elle reçoit une primitive et applique la logique de chaînage ou de traitement des blocs, sans jamais connaître les détails internes de l'algorithme qu'elle encapsule.

```
from abc import ABC, abstractmethod
from domain.cipher.CipherPrimitive import CipherPrimitive

class OperationMode(ABC):
    """Interface commune pour les modes d'opération de chiffrement par blocs."""

    def __init__(self, primitive: CipherPrimitive) -> None:
        self._primitive = primitive

    @property
    def primitive(self) -> CipherPrimitive:
        return self._primitive

    @abstractmethod
    def encrypt(self, plaintext: bytes, **kwargs) -> bytes:
        """Chiffre un texte en clair de longueur arbitraire."""

    @abstractmethod
    def decrypt(self, ciphertext: bytes, **kwargs) -> bytes:
        """Déchiffre un texte chiffré de longueur arbitraire."""
```

Code source complet : [domain/mode/OperationMode.py](#)

Extrait 3 — EncryptionEngine

Ce composant assure la composition d'une primitive et d'un mode, constituant le service de chiffrement complet. Il valide que le mode et la primitive sont bien liés, garantissant la cohérence des invariants du domaine.

```
from domain.cipher.CipherPrimitive import CipherPrimitive
from domain.mode.OperationMode import OperationMode

class EncryptionEngine:
    def __init__(self, primitive: CipherPrimitive,
                 mode: OperationMode) -> None:
        if mode.primitive is not primitive:
            raise ValueError(
                "The mode's primitive must be the same object as the engine's "
                "primitive. Build the mode with the primitive first, then pass "
                "both to EncryptionEngine."
            )
        self._primitive = primitive
        self._mode = mode

    @property
    def primitive(self) -> CipherPrimitive:
        return self._primitive

    @property
    def mode(self) -> OperationMode:
        return self._mode

    def encrypt(self, plaintext: bytes, **kwargs) -> bytes:
        return self._mode.encrypt(plaintext, **kwargs)

    def decrypt(self, ciphertext: bytes, **kwargs) -> bytes:
        return self._mode.decrypt(ciphertext, **kwargs)
```

Code source complet : [domain/engine/EncryptionEngine.py](#)

7.2 Validation fonctionnelle effectuée

La stratégie de validation fonctionnelle a été intégralement définie et documentée. Elle repose sur l'utilisation systématique de vecteurs de tests standards (Known Answer Tests), essentiels pour vérifier la conformité des résultats produits par les implémentations cryptographiques.

Ce protocole de validation s'articule autour de deux axes de vérification :

1. Conformité des algorithmes : les primitives cryptographiques sont confrontées aux valeurs de référence des standards reconnus, soit le FIPS 197 pour l'AES, le SP 800-17 du NIST pour le DES, le SP 800-67 pour le 3DES, et la RFC 8439 pour ChaCha20. Pour le 3DES, la validation confirme spécifiquement le respect des trois opérations successives de chiffrement.

2. Exactitude des modes : la conformité des modes par blocs (ECB, CBC, CTR) est validée selon le SP 800-38A du NIST, le mode GCM selon le SP 800-38D, et StreamMode est couvert par la validation RFC 8439 de ChaCha20.

Ce protocole de validation constitue un prérequis indispensable aux campagnes expérimentales. Cette rigueur méthodologique garantit que les mesures de performance temporelle et les analyses de robustesse sécuritaire (diffusion par substitution et confusion par permutation) reposent sur un socle logiciel certifié mathématiquement, assurant ainsi la crédibilité et la reproductibilité des résultats finaux.

7.3 Premières expérimentations

Les premières campagnes expérimentales ont été initiées sur la plateforme Windows (x86-64), suivant une approche de développement incrémental. Elles constituent une phase de tests d'intégration du système dont les objectifs sont les suivants :

1. Validation de la chaîne d'exécution : vérifier la collaboration entre les scripts de lancement, le contrôleur d'expérimentation et le moteur de chiffrement afin d'assurer un flux de données sans interruption.
2. Vérification de la portabilité : confirmer que l'artefact logiciel s'exécute de manière identique sur l'ordinateur personnel et sur le Raspberry Pi, garantissant ainsi la neutralité du cadre expérimental.
3. Fiabilité de la collecte : s'assurer que le bloc d'analyse génère correctement les fichiers de résultats structurés, en distinguant les mesures de performance temporelle (débit) des indicateurs de robustesse sécuritaire (effet d'avalanche).
4. Validation des indicateurs de sécurité : mesurer l'effet d'avalanche pour chaque primitive, conformément au protocole de distance de Hamming établi, afin d'évaluer les propriétés de diffusion et de confusion intrinsèques aux algorithmes.

Ces essais préliminaires sécurisent la phase de collecte systématique des mesures, indispensable pour valider ou infirmer les hypothèses de recherche lors des étapes ultérieures du projet.

7.4 Difficultés rencontrées et ajustements

Les principales difficultés identifiées concernent la traduction du design tactique en une implémentation logicielle qui préserve la pureté des abstractions du domaine. L'enjeu majeur réside dans l'intégration homogène d'algorithmes issus de sources hétérogènes (bibliothèque PyCryptodome pour la majorité des primitives, implémentation externe via ctypes pour Twofish) ainsi que dans l'adaptation de ChaCha20, chiffrement par flux, au sein d'une architecture conçue principalement autour des chiffrements par blocs. Ces

contraintes exigent une gestion rigoureuse des interfaces pour éviter toute forme de couplage excessif.

Ces défis ont été anticipés dès la phase de modélisation par les ajustements suivants :

1. Réduction de la complexité accidentelle : l'architecture privilégie la composition des objets afin de garantir que chaque module possède une responsabilité unique, limitant ainsi la propagation d'erreurs lors de l'évolution du code.
2. Uniformisation des paramètres : la définition de contrats d'interface clairs permet une gestion standardisée des vecteurs d'initialisation et des tailles de blocs, assurant la comparabilité rigoureuse des mesures de performance.
3. Garantie de neutralité : le choix d'une structure modulaire agnostique de l'algorithme répond directement au besoin de neutralité du cadre expérimental. Cette approche assure que les résultats obtenus reflètent les propriétés cryptographiques intrinsèques et non des biais structurels du moteur de chiffrement.

Cette démarche proactive de gestion des risques sécurise la transition vers les phases de production de données et garantit la faisabilité du projet dans le respect des délais académiques.

8. Conclusion du rapport de mi-parcours

Ce rapport de mi-parcours a permis de formaliser la conception logicielle du moteur expérimental destiné à la comparaison rigoureuse des algorithmes de chiffrement symétrique. À partir de l'analyse du domaine cryptographique et des objectifs méthodologiques définis lors de la phase de planification, une architecture modulaire et cohérente a été proposée afin d'isoler strictement les primitives cryptographiques, les modes d'opération et les mécanismes d'orchestration expérimentale.

L'utilisation conjointe de modèles UML sélectionnés de manière ciblée a permis d'articuler les vues conceptuelle, structurelle et dynamique du système, tout en garantissant la neutralité des mesures et la reproductibilité des expérimentations. La distinction explicite entre comportement nominal et protocole de test de performance illustre en particulier l'attention portée à la séparation entre logique cryptographique et mécanismes de mesure.

Les extraits de code présentés dans ce rapport témoignent de la solidité des abstractions retenues et de leur capacité à accueillir l'ensemble des primitives et modes d'opération du système, y compris les chiffrements par flux.

Cette phase de conception établit ainsi une assise technique solide pour les campagnes expérimentales à venir, visant à analyser les performances et les propriétés de robustesse sécuritaire des algorithmes étudiés, conformément aux objectifs du projet.

Bibliographie

- (NIST), National Institute of Standards and Technology. (1998). *Modes of Operation Validation System (MOVS): Requirements and Procedures (SP 800-17)*. (NIST, Ed.) Retrieved from <https://csrc.nist.gov/publications/detail/sp/800-17/final>
- (NIST), National Institute of Standards and Technology. (2001). *Recommendation for Block Cipher Modes of Operation: Methods and Techniques (SP 800-38A)*. (NIST, Ed.) Retrieved from <https://doi.org/10.6028/NIST.SP.800-38A>
- (NIST), National Institute of Standards and Technology. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC (SP 800-38D)*. (NIST, Ed.) Retrieved from <https://doi.org/10.6028/NIST.SP.800-38D>
- (NIST), National Institute of Standards and Technology. (2020). *Guideline for Using Cryptographic Standards in the Federal Government (SP 800-175Br1)*. (NIST, Ed.) Retrieved from <https://doi.org/10.6028/NIST.SP.800-175Br1>
- (NIST), National Institute of Standards and Technology. (2020). *Recommendation for Key Management (SP 800-57 Part 1 Rev. 5)*. (NIST, Ed.) Retrieved from <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- (NIST), National Institute of Standards and Technology. (2023). *Advanced Encryption Standard (AES) (FIPS 197, mise à jour 1)*. (NIST, Ed.) Retrieved from <https://doi.org/10.6028/NIST.FIPS.197-upd1>
- (NIST), National Institute of Standards and Technology. (2024). *Hamming distance*. (NIST, Ed.) Retrieved from <https://xlinux.nist.gov/dads/HTML/HammingDistance.html>
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. (Addison-Wesley, Ed.)
- Fowler, M. (2003, 2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language (3e éd.)*. (Addison-Wesley, Ed.)
- Katz, J., & Lindell, Y. (2007). *Introduction to Modern Cryptography*. (C. Press, Ed.)
- Khononov, V. (2021). *Learning Domain-Driven Design*. (O. Media, Ed.)
- Paar, C., & Pelzl, J. (2010). *Understanding Cryptography*. (Springer, Ed.)

Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice (7e éd.)*.
(Pearson, Ed.)

Annexe

Annexe A – Structure des dossiers et fichiers

Le projet est structuré autour d'un répertoire principal intitulé `crypto-experiments`, qui constitue la racine du dépôt. Ce répertoire regroupe l'ensemble des composants nécessaires à l'implémentation, à l'exécution et à l'analyse des expérimentations de chiffrement symétrique.

La structure du projet repose sur une séparation claire des responsabilités, distinguant le domaine applicatif, l'orchestration des expérimentations, les scripts d'exécution, ainsi que les données produites. Chaque dossier principal correspond à un rôle précis dans l'architecture du système.

La présente annexe a pour objectif de détailler l'organisation des dossiers et des fichiers du projet, ainsi que leur utilité respective. Pour chaque composant, sa fonction et son implication dans le fonctionnement global du système sont précisées afin de faciliter la compréhension de l'implémentation.

A.1 `domain/`

Le dossier **`domain/`** regroupe l'ensemble des composants constituant le cœur logique du système. Il représente les concepts fondamentaux liés au domaine du chiffrement symétrique et définit *ce que le système est*, indépendamment de son exécution, de l'orchestration des expérimentations ou des mécanismes de persistance des données.

Les classes contenues dans ce dossier implémentent les abstractions du domaine, notamment les primitives cryptographiques, les modes d'opération et leur composition au sein d'un moteur de chiffrement. Ces composants sont conçus pour être autonomes et réutilisables, et ne dépendent ni des scripts d'exécution ni des formats de sortie des résultats.

La structure interne du dossier `domain/` est organisée en sous-dossiers, chacun correspondant à une famille cohérente de concepts du domaine. Les sections suivantes détaillent la structure et le rôle de chacun de ces sous-dossiers.

Tableau A.1 — Sous-dossier cipher/ (Primitives cryptographiques)

Élément	Type	Description
cipher/	Dossier	Contient les primitives cryptographiques du domaine
CipherPrimitive.py	Fichier	Classe abstraite définissant l'interface commune à toutes les primitives cryptographiques (opérations de chiffrement et de déchiffrement sur des blocs)
AES.py	Fichier	Implémentation concrète de l'algorithme AES
DES.py	Fichier	Implémentation concrète de l'algorithme DES
TripleDES.py	Fichier	Implémentation de la variante Triple DES
Twofish.py	Fichier	Implémentation concrète de l'algorithme Twofish
ChaCha20.py	Fichier	Implémentation concrète de l'algorithme ChaCha20

Tableau A.2 — Sous-dossier mode/ (Modes d'opération)

Élément	Type	Description
mode/	Dossier	Contient les modes d'opération applicables aux primitives cryptographiques
OperationMode.py	Fichier	Classe abstraite définissant le comportement général d'un mode d'opération
ECB.py	Fichier	Implémentation du mode ECB (Electronic Codebook)
CBC.py	Fichier	Implémentation du mode CBC (Cipher Block Chaining)
CTR.py	Fichier	Implémentation du mode CTR (Counter)
GCM.py	Fichier	Implémentation du mode GCM (Galois/Counter Mode)
StreamMode.py	Fichier	Mode pass-through pour les chiffrements par flot

Tableau A.3 — Sous-dossier engine/ (Moteur de chiffrement)

Élément	Type	Description
engine/	Dossier	Contient le moteur de chiffrement du domaine

EncryptionEngine.py	Fichier	Classe responsable de la composition d'une primitive cryptographique et d'un mode d'opération afin de fournir un service de chiffrement de messages
---------------------	---------	---

Alors que le dossier `domain/` décrit les concepts fondamentaux et la logique métier du système, le dossier `application/` regroupe les composants responsables de l'orchestration et de la mise en œuvre de ces abstractions lors de l'exécution des expérimentations.

A.2 Dossier `application/`

Le dossier **`application/`** regroupe les composants responsables de l'orchestration des expérimentations. Il constitue l'interface entre le cœur logique du système (le domaine) et les mécanismes d'exécution déclenchés par les scripts.

Contrairement au dossier `domain/`, ce dossier ne contient pas de logique cryptographique. Il se concentre exclusivement sur l'organisation des appels, la configuration des expériences et la coordination des composants du domaine, sans enfreindre la séparation des responsabilités définie lors de la conception.

Les classes présentes dans ce dossier utilisent les abstractions du domaine afin de piloter leur exécution, mais ne dépendent ni des détails d'implémentation des algorithmes ni des mécanismes de persistance des résultats.

Tableau A.4 — Dossier `application/` (Orchestration des expérimentations)

Élément	Type	Description
<code>application/</code>	Dossier	Contient les composants d'orchestration du système
<code>ExperimentController.py</code>	Fichier	Classe responsable de la coordination des expérimentations, incluant le pilotage des moteurs de chiffrement, le chronométrage des opérations et la collecte des résultats.

Les mécanismes d'orchestration décrits ci-dessus sont activés par des scripts d'exécution, présentés dans la section suivante, qui constituent les points d'entrée du système.

A.3 Dossier scripts/

Le dossier **scripts/** regroupe les points d'entrée du système, responsables du lancement effectif des expérimentations. Les scripts contenus dans ce dossier constituent l'interface entre l'utilisateur et l'application, en déclenchant l'exécution des mécanismes d'orchestration définis dans le dossier application/.

Ces scripts ne contiennent aucune logique métier ni cryptographique. Leur rôle se limite à l'initialisation du programme, à la configuration des paramètres nécessaires et à l'appel des composants applicatifs appropriés. Cette séparation permet de préserver l'indépendance du domaine et de l'orchestration vis-à-vis des modalités d'exécution.

Tableau A.5 — Dossier scripts/ (Points d'entrée du système)

Élément	Type	Description
scripts/	Dossier	Contient les scripts permettant de lancer l'exécution du système
experiment.py	Fichier	Script principal servant de point d'entrée, responsable de la configuration des moteurs de chiffrement, du déclenchement des expérimentations via l'ExperimentController et de la persistance des résultats en CSV.

Les scripts présentés ci-dessus génèrent des données de sortie à l'issue de l'exécution des expérimentations. Ces résultats sont stockés et organisés dans le dossier data/, décrit dans la section suivante.

A.4 Dossier data/

Le dossier **data/** regroupe les données produites par l'exécution des expérimentations. Il contient les résultats générés par le système sous forme de fichiers structurés, destinés à une analyse ultérieure à l'aide d'outils externes (tableurs, scripts d'analyse, etc.).

Ce dossier ne fait pas partie du cœur logique du système et n'intervient pas dans le processus de chiffrement. Les données qu'il contient sont générées à l'issue des expérimentations et ne sont pas utilisées comme données d'entrée par le domaine applicatif, ce qui permet de maintenir une séparation claire entre la logique métier et la persistance des résultats.

Tableau A.6 — Dossier data/ (Résultats expérimentaux)

Élément	Type	Description
data/	Dossier	Contient les données produites lors de l'exécution des expérimentations
results/	Dossier	Sous-dossier regroupant les fichiers de résultats
experiment_*.csv	Fichier	Fichiers CSV contenant les mesures et les résultats des expérimentations de chiffrement

Note : La structure du dossier data/ est conçue pour accueillir des sous-dossiers supplémentaires (visualisations graphiques, etc.) lors des phases ultérieures du projet.

L'organisation présentée dans cette annexe illustre la séparation claire des responsabilités adoptée dans le projet, permettant de distinguer le domaine, l'orchestration, l'exécution et les résultats expérimentaux.

A.5 Dossier validation/

Le dossier validation/ regroupe les composants responsables de la vérification fonctionnelle des primitives cryptographiques et des modes d'opération. Il contient les vecteurs de tests standards (Known Answer Tests) utilisés pour confirmer la conformité des implémentations aux standards reconnus avant toute mesure de performance.

Tableau A.7 — Dossier validation/ (Validation fonctionnelle)

Élément	Type	Description
validation/	Dossier	Contient les tests de validation fonctionnelle du système
kat_aes.py	Fichier	Vecteurs de tests NIST pour la validation de l'AES (FIPS 197)
kat_des.py	Fichier	Vecteurs de tests NIST pour la validation du DES (SP 800-17)
kat_3des.py	Fichier	Vecteurs de tests NIST pour la validation du 3DES (SP 800-67)
kat_modes.py	Fichier	Vecteurs de tests pour la validation des modes d'opération (SP 800-38A)
kat_gcm.py	Fichier	Vecteurs de tests pour la validation du mode GCM (SP 800-38D)
Kat_chacha20.py	Fichier	Vecteurs de tests pour la validation de ChaCha20 (RFC 8439)

Annexe B – Justification des relations du diagramme de classes

Tableau B.1 — Relations UML entre les classes principales

Classe source	Classe cible	Type de relation UML	Multiplicité
ExperimentController	EncryptionEngine	Association	1 — 1
EncryptionEngine	OperationMode	Composition	1 — 1
OperationMode	CipherPrimitive	Association	1 — 1
EncryptionEngine	CipherPrimitive	Composition	1 — 1

Tableau B.2 — Relations d'héritage (généralisation)

Classe abstraite	Classe concrète	Type
CipherPrimitive	AES	Héritage
CipherPrimitive	DES	Héritage
CipherPrimitive	TripleDES	Héritage
CipherPrimitive	Twofish	Héritage
CipherPrimitive	ChaCha20	Héritage
OperationMode	ECB	Héritage
OperationMode	CBC	Héritage
OperationMode	CTR	Héritage
OperationMode	GCM	Héritage
OperationMode	StreamMode	Héritage

B.1 Justification détaillée des relations

1. ExperimentController — EncryptionEngine (Association, 1 — 1)

ExperimentController est associé à exactement un EncryptionEngine qu'il orchestre pour exécuter les scénarios expérimentaux. Cette relation traduit le rôle d'orchestration du contrôleur, qui coordonne la configuration de chiffrement sans en posséder le cycle de vie. La multiplicité 1 — 1 indique qu'une instance de contrôleur est configurée pour un unique moteur à la fois, ce qui exclut une relation de composition.

2. EncryptionEngine — OperationMode (Composition, 1 — 1)

EncryptionEngine est en relation de composition avec OperationMode, le mode d'opération constituant un élément indispensable et non partagé de la configuration du moteur de chiffrement. Le moteur ne peut fonctionner sans mode, et le mode n'est jamais utilisé indépendamment du moteur qui l'emploie. La multiplicité 1 — 1 reflète le fait que chaque moteur est configuré avec exactement un mode actif à la fois, et que le cycle de vie du mode est entièrement contrôlé par celui du moteur. Cette relation correspond à une dépendance forte de type « tout-partie », justifiant l'utilisation d'une composition UML.

3. OperationMode — CipherPrimitive (Association, 1 — 1)

OperationMode est associé à une CipherPrimitive qu'il utilise pour appliquer la logique de chaînage sur les blocs chiffrés. Bien que la relation soit de multiplicité 1 — 1, la primitive cryptographique demeure conceptuellement indépendante du mode d'opération et conserve son propre cycle de vie. La primitive représente l'algorithme cryptographique sous-jacent, tandis que le mode implémente uniquement la stratégie de traitement des blocs. Cette relation correspond donc à une collaboration fonctionnelle et non à une relation de possession, ce qui justifie l'utilisation d'une association simple plutôt qu'une composition.

4. EncryptionEngine — Cipher Primitive (Association, 1 — 1)

EncryptionEngine est en relation de composition avec CipherPrimitive, la primitive constituant un élément constitutif indissociable du moteur de chiffrement. Le moteur ne peut fonctionner sans primitive définie, et la primitive référencée n'est jamais partagée avec un autre moteur. La multiplicité 1 — 1 reflète qu'un moteur emploie exactement une primitive à la fois. Cette relation matérialise également une dépendance de validation (Design by Contract) : l'EncryptionEngine vérifie à la construction que le mode reçu encapsule bien cette même instance de primitive, garantissant la cohérence des invariants du domaine.

B.2 Justification des relations d'héritage (généralisation)

CipherPrimitive est modélisée comme une classe abstraite représentant le concept général de primitive de chiffrement symétrique. Les classes AES, DES, TripleDES, Twofish et ChaCha20 constituent des spécialisations concrètes de cette abstraction. La relation d'héritage permet de capturer les variantes algorithmiques tout en garantissant leur substituabilité au sein du système. Grâce à cette généralisation, les composants qui manipulent des primitives cryptographiques peuvent reposer sur une interface commune, indépendamment de l'algorithme effectivement utilisé. Cette modélisation favorise l'extensibilité du système et permet l'ajout de nouvelles primitives sans modification de la logique existante.

De même, OperationMode est modélisée comme une classe abstraite définissant le comportement commun à tous les modes d'opération. Les classes ECB, CBC, CTR, GCM et StreamMode en sont les spécialisations concrètes. Chaque mode implémente sa propre logique de chaînage ou de traitement des blocs tout en respectant le contrat d'interface défini par la classe parente, garantissant leur substituabilité au sein du moteur de chiffrement.